2. Analyze the two GenAI programs. The analysis must include:

a. Name of two GenAI programs

Gemini 3.6 Flash & Claude Sonnet 5.5

b. How you accessed them (e.g., Google query, CoPilot on my computer)

I accessed both GenAI programs through their websites (claude.ai & gemini.google.com respectively)

c. Prompt used. (You must use the same prompt for both GenAIs.)

Generate C++ code using objects, not functions, to implement the following requirements:

- Sudoku is a number puzzle in which a 9x9 grid is subdivided into 9 3x3 sub-grids, or blocks.
- A typical puzzle is shown here (image credit: Wikimedia Commons):
- Each of the cells in the puzzle must be filled in with a single digit 1-9, subject to the following constraints:
- o Each digit from 1 to 9 must appear exactly once in each row.
- o Each digit from 1 to 9 must appear exactly once in each column.
- o Each digit from 1 to 9 must appear exactly once in each block.
- Published puzzles are usually constructed to have a unique solution. This requires that at least 17 cells be filled in (and may require more; depending on the puzzle and which cells are empty, as many as 78 filled cells can be required to guarantee uniqueness). In general, the number of cells that are already filled in is not a particularly good indicator of how difficult a Sudoku puzzle is to solve.
- The Sudoku problem, then, is this: Given a partially completed Sudoku puzzle, find the (or at least a) solution; if no solution exists, prove that.
- It is possible to solve a Sudoku puzzle entirely through logic. For example, in the above puzzle, consider the bottom right block, top right cell. Because of other numbers in the same row, column, or grid, that number cannot be 1, 2, 3, 5, 6, 7, 8, or 9, and so must be 4. By proceeding in this fashion, using these and more advanced deduction rules, it's possible to gradually fill in the entire grid.
- That is not the approach we're going to use. Instead, we're going to implement a brute-force search, in which we exhaustively try possibilities until we find a solution. This will be slower to run but is easier to solve. (And by "slower to run," we mean it'll finish in a few tenths of a second rather than a few milliseconds.)
- Here's the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates report no solution, which again implies we took a wrong path somewhere earlier).

- This approach, by the way, is called depth-first search with backtracking. For each empty cell we make an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find something else to try, and so on. If a solution exists, we are guaranteed to find it, because we exhaustively try all possibilities.
- You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.
- Write a recursive object-oriented program to find the solution to the five puzzle files.
- The output for each solution, should include:
- o The puzzle file name (e.g., puzzle1.txt)
- o The puzzle from the file printed out.
- o The solution to the puzzle printed out or if no solution is found, “No solution found” printed out.
- The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out.
- To help you debug your program, solution1.txt is a solution to puzzle1.txt.

HERE IS THE solution.txt :

```
5 3 9 8 2 4 1 7 6
1 8 6 5 7 9 2 4 3
4 7 2 1 3 6 8 9 5
9 2 4 3 8 7 5 6 1
3 1 8 6 9 5 4 2 7
6 5 7 4 1 2 3 8 9
7 4 5 2 6 1 9 3 8
8 9 1 7 4 3 6 5 2
2 6 3 9 5 8 7 1 4
Solutions found: 1
```

HERE IS puzzle.1:

```
5 _ _ _ _ _ 1 7 _
1 _ 6 5 _ 9 _ 4 _
4 7 2 1 _ 6 _ _ _
9 _ _ _ _ _ 5 _ _
_ 1 8 _ 9 5 4 _ _
6 _ _ 4 _ 2 3 8 9
_ 4 _ _ _ _ 9 3 _
_ 9 _ 7 _ 3 _ 5 _
2 6 3 9 5 8 7 1 4
```

HERE IS puzzle2.txt:

```
5 3 _ 8 _ 4 _ 7 6
```

1 _ 6 _ 7 9 _ 4 3
_ 7 _ _ 3 6 _ _ 5
9 2 4 _ 8 _ 5 6 _
3 _ 8 6 9 _ 4 2 7
_ 5 _ 4 1 2 3 _ _
7 4 5 _ 6 _ 9 _ 8
8 _ 1 7 _ 3 6 5 2
_ 6 3 9 5 _ 7 _ 4

HERE IS puzzle3.txt:
7 _ _ 3 _ 6 _ 4 _
3 4 _ 5 _ 9 6 _ 8
6 1 9 8 _ 7 5 2 3
4 9 _ _ 8 5 _ _ 7
1 2 _ _ _ _ 3 6 5
_ 7 6 _ 3 _ 8 _ _
2 _ 1 4 9 _ _ _ 6
_ 3 _ 2 _ 8 4 _ 1
8 6 4 7 _ _ 9 3 2

HERE IS puzzle4.txt:
7 3 2 _ 8 4 6 9 1
9 1 _ 3 _ _ 5 2 _
8 _ _ 9 _ 2 7 3 4
5 4 9 _ _ _ 8 6 3
1 _ _ _ 3 _ 2 _ 7
_ 2 3 _ 4 8 9 1 _
3 9 _ 8 5 _ 4 7 2
_ 7 _ 4 _ 3 _ _ 6
4 6 8 _ 7 _ _ 5 9

HERE IS puzzle5.txt:
_ _ _ _ _ 4 2 _ 1
_ _ _ _ 7 _ _ _ 5
_ _ 8 1 _ 5 7 _ _
_ 4 1 _ 3 2 8 _ _
3 8 9 _ 5 6 1 2 7
2 _ _ _ _ 8 3 _ _
_ 2 4 _ _ 7 _ 1 _
8 3 6 _ 9 _ 4 _ _
_ _ 7 _ _ 3 5 _ _

d. Generated code from both GenAIs.

GEMINI'S GENERATION:

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>

class SudokuBoard
{
private:
   int grid[9][9];

   // Check if placing a number is valid according to Sudoku rules
   bool isValid(int row, int col, int num) const
   {
       // Check row
       for (int c = 0; c < 9; ++c)
       {
           if (grid[row][c] == num)
           {
               return false;
           }
       }

       // Check column
       for (int r = 0; r < 9; ++r)
       {
           if (grid[r][col] == num)
           {
               return false;
           }
       }

       // Check 3x3 sub-grid / block
       int startRow = row - row % 3;
       int startCol = col - col % 3;
       for (int r = 0; r < 3; ++r)
       {
           for (int c = 0; c < 3; ++c)
           {
               if (grid[startRow + r][startCol + c] == num)
               {
                   return false;
```

```
            }
        }
    }

    return true;
}

// Find the next empty cell (represented by 0)
bool findEmptyCell(int &row, int &col) const
{
    for (row = 0; row < 9; ++row)
    {
        for (col = 0; col < 9; ++col)
        {
            if (grid[row][col] == 0)
            {
                return true;
            }
        }
    }
    return false;
}

// Recursive backtracking method
void solveRecursive(int &solutionCount)
{
    int row, col;

    // If no empty cell remains, a solution is found
    if (!findEmptyCell(row, col))
    {
        solutionCount++;
        std::cout << "\nSolution " << solutionCount << ":\n";
        printBoard();
        return;
    }

    // Try digits 1 through 9
    for (int num = 1; num <= 9; ++num)
    {
        if (isValid(row, col, num))
        {
```

```
                grid[row][col] = num; // Place candidate

                solveRecursive(solutionCount); // Recursive step

                grid[row][col] = 0; // Backtrack
            }
        }
    }

public:
    SudokuBoard()
    {
        for (int i = 0; i < 9; ++i)
        {
            for (int j = 0; j < 9; ++j)
            {
                grid[i][j] = 0;
            }
        }
    }

    // Load puzzle from file
    bool loadFromFile(const std::string &filename)
    {
        std::ifstream file(filename);
        if (!file.is_open())
        {
            return false;
        }

        char ch;
        for (int i = 0; i < 9; ++i)
        {
            for (int j = 0; j < 9; ++j)
            {
                if (!(file >> ch))
                {
                    return false;
                }
                if (ch == '_')
                {
                    grid[i][j] = 0;
```

```
            }
            else if (ch >= '1' && ch <= '9')
            {
                grid[i][j] = ch - '0';
            }
            else
            {
                grid[i][j] = 0;
            }
        }
    }
    return true;
}

// Print the board state
void printBoard() const
{
    for (int i = 0; i < 9; ++i)
    {
        for (int j = 0; j < 9; ++j)
        {
            if (grid[i][j] == 0)
            {
                std::cout << "_";
            }
            else
            {
                std::cout << grid[i][j];
            }
            if (j < 8)
            {
                std::cout << " ";
            }
        }
        std::cout << "\n";
    }
}

// Public interface to solve the puzzle
void solve()
{
    int solutionCount = 0;
```

```
        solveRecursive(solutionCount);
        if (solutionCount == 0)
        {
            std::cout << "No solution found\n";
        }
        else
        {
            std::cout << "\nSolutions found: " << solutionCount << "\n";
        }
    }
};

int main()
{
    std::string filenames[] = {"puzzle1.txt", "puzzle2.txt", "puzzle3.txt",
"puzzle4.txt", "puzzle5.txt"};

    for (const auto &filename : filenames)
    {
        std::cout << "========================================\n";
        std::cout << "Puzzle File: " << filename << "\n";
        std::cout << "========================================\n";

        SudokuBoard puzzle;
        if (puzzle.loadFromFile(filename))
        {
            std::cout << "Initial Puzzle:\n";
            puzzle.printBoard();
            std::cout << "\n--- Solving ---\n";
            puzzle.solve();
        }
        else
        {
            std::cout << "Error: Could not open file " << filename << "\n";
        }
        std::cout << "========================================\n\n";
    }

    return 0;
}
```

CLAUDE'S GENERATION:

```cpp
#include <fstream>
#include <iostream>
#include <string>
#include <vector>

// ---------------------------------------------------------------
// Board: holds a 9x9 Sudoku grid (0 = empty cell)
// ---------------------------------------------------------------
class Board
{
public:
   static const int SIZE = 9;
   static const int BLOCK = 3;

   Board()
   {
       for (int r = 0; r < SIZE; ++r)
           for (int c = 0; c < SIZE; ++c)
               cells[r][c] = 0;
   }

   // Reads a puzzle file. Digits 1-9 are givens, '_' is a blank.
   bool loadFromFile(const std::string &fileName)
   {
       std::ifstream in(fileName);
       if (!in)
           return false;

       int count = 0;
       char ch;
       while (count < SIZE * SIZE && in >> ch)
       {
           if (ch == '_')
           {
               cells[count / SIZE][count % SIZE] = 0;
               ++count;
           }
           else if (ch >= '1' && ch <= '9')
           {
               cells[count / SIZE][count % SIZE] = ch - '0';
               ++count;
           }
```

```
            // any other character is ignored
        }
        return count == SIZE * SIZE;
    }

    int get(int row, int col) const { return cells[row][col]; }
    void set(int row, int col, int value) { cells[row][col] = value; }

    // Can 'value' legally go in (row, col)? Checks row, column and block.
    bool canPlace(int row, int col, int value) const
    {
        for (int i = 0; i < SIZE; ++i)
        {
            if (cells[row][i] == value)
                return false;
            if (cells[i][col] == value)
                return false;
        }
        int startRow = (row / BLOCK) * BLOCK;
        int startCol = (col / BLOCK) * BLOCK;
        for (int r = startRow; r < startRow + BLOCK; ++r)
            for (int c = startCol; c < startCol + BLOCK; ++c)
                if (cells[r][c] == value)
                    return false;
        return true;
    }

    // Checks that the given digits don't already break the rules.
    bool givensAreValid()
    {
        for (int r = 0; r < SIZE; ++r)
        {
            for (int c = 0; c < SIZE; ++c)
            {
                int v = cells[r][c];
                if (v == 0)
                    continue;
                cells[r][c] = 0; // temporarily remove itself
                bool ok = canPlace(r, c, v);
                cells[r][c] = v;
                if (!ok)
                    return false;
```

```
        }
    }
    return true;
}

// Finds the next empty cell. Returns false if the board is full.
bool findEmpty(int &row, int &col) const
{
    for (row = 0; row < SIZE; ++row)
        for (col = 0; col < SIZE; ++col)
            if (cells[row][col] == 0)
                return true;
    return false;
}

// Builds the list of candidate digits for an empty cell.
std::vector<int> candidates(int row, int col) const
{
    std::vector<int> list;
    for (int v = 1; v <= SIZE; ++v)
        if (canPlace(row, col, v))
            list.push_back(v);
    return list;
}

void print(std::ostream &out) const
{
    for (int r = 0; r < SIZE; ++r)
    {
        for (int c = 0; c < SIZE; ++c)
        {
            if (c > 0)
                out << ' ';
            if (cells[r][c] == 0)
                out << '_';
            else
                out << cells[r][c];
        }
        out << '\n';
    }
}
```

```
private:
   int cells[SIZE][SIZE];
};

// ---------------------------------------------------------------
// SudokuSolver: recursive DFS with backtracking, collects every
// solution it finds.
// ---------------------------------------------------------------
class SudokuSolver
{
public:
   explicit SudokuSolver(const Board &puzzle) : board(puzzle) {}

   // Runs the search; returns the number of solutions found.
   int solve()
   {
       solutions.clear();
       if (board.givensAreValid())
           search();
       return static_cast<int>(solutions.size());
   }

   const std::vector<Board> &getSolutions() const { return solutions; }

private:
   Board board; // working copy that gets filled in
   std::vector<Board> solutions;

   void search()
   {
       int row, col;

       // No empty cell left: all 81 cells are filled -> a solution.
       if (!board.findEmpty(row, col))
       {
           solutions.push_back(board);
           return;
       }

       // No candidates means a wrong turn earlier; just return
       // (the loop below simply doesn't run).
       std::vector<int> options = board.candidates(row, col);
```

```
        for (int value : options)
        {
            board.set(row, col, value); // try it
            search();                   // recurse with k+1 filled
            board.set(row, col, 0);     // backtrack
        }
    }
};

// ----------------------------------------------------------------
// PuzzleRunner: loads one file, solves it, prints the report.
// ----------------------------------------------------------------
class PuzzleRunner
{
public:
    explicit PuzzleRunner(const std::string &file) : fileName(file) {}

    void run(std::ostream &out) const
    {
        out << "================================\n";
        out << fileName << '\n';
        out << "================================\n";

        Board puzzle;
        if (!puzzle.loadFromFile(fileName))
        {
            out << "Could not read a valid 9x9 puzzle from " << fileName << "\n\n";
            return;
        }

        out << "Puzzle:\n";
        puzzle.print(out);
        out << '\n';

        SudokuSolver solver(puzzle);
        int count = solver.solve();

        if (count == 0)
        {
            out << "No solution found\n\n";
            return;
        }
```

```
        const std::vector<Board> &sols = solver.getSolutions();
        for (int i = 0; i < count; ++i)
        {
            out << "Solution " << (i + 1) << ":\n";
            sols[i].print(out);
            out << '\n';
        }
        out << "Solutions found: " << count << "\n\n";
    }

private:
    std::string fileName;
};

int main()
{
    const std::vector<std::string> files = {
        "puzzle1.txt", "puzzle2.txt", "puzzle3.txt",
        "puzzle4.txt", "puzzle5.txt"};

    for (const std::string &f : files)
    {
        PuzzleRunner runner(f);
        runner.run(std::cout);
    }
    return 0;
}
```

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the "GenAI Code Comparison" lecture.

Correctness: Both Claude's and Gemini's code use recursive backtracking to solve Sudoku puzzles by checking whether each number follows the row, column, and 3x3 block rules. However, Claude's code is more reliable because it checks whether the original puzzle contains conflicting numbers before solving it. Gemini's code does not check whether the original puzzle is valid, so it could incorrectly accept a completed but invalid puzzle as a solution. Claude's code also checks whether the puzzle file contains exactly 81 valid cells, while Gemini's code only checks whether it can read 81 characters.

Execution Time: Both programs have a worst-case exponential running time because they use brute-force search and try different combinations of numbers until they find all possible solutions. Claude's code creates a list of possible candidates for each empty cell before trying them, while Gemini's code checks each number from 1 to 9 individually.

Both use depth-first search with backtracking, but neither uses more advanced techniques to reduce the number of possibilities that need to be checked. Since both programs find every solution, their execution time depends on the difficulty of the puzzle and the number of solutions.

Space Complexity: Both programs use a 9x9 grid and recursive calls, so the board itself requires constant space because its size never changes. The recursion stack can grow with the number of empty cells, making its space usage proportional to the number of empty cells. Claude's code also stores every solution in a vector, which can use a significant amount of additional memory if a puzzle has many solutions. Gemini's code prints each solution immediately instead of storing them, so it generally uses less memory when handling multiple solutions.

Maintainability: Claude's code is easier to maintain because it separates the program into three classes: Board, SudokuSolver, and PuzzleRunner. Each class has a specific responsibility, making the code easier to understand, test, and modify. Gemini's code combines most of the solving and printing operations in the SudokuBoard class, which makes it less modular. Claude's code also separates output handling from the solving process, while Gemini's solver prints directly to the console. This makes Claude's code more flexible if the output format needs to change.

f. Select one to use as the basis for this assignment and justify why you picked it.

I would use Claude's code as the basis for this assignment because it is better organized and checks whether the original puzzle is valid before solving it. Its separate classes make the code easier to understand and modify, and storing the solutions allows them to be printed after the search is complete. Even though Gemini's code uses less memory by printing solutions immediately, I think Claude's code is a better starting point because it is more modular and easier to improve.

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

Correctness: I improved the file validation so the program rejects invalid characters, extra data, files that do not contain exactly 81 cells, and puzzles with conflicting numbers. The improved program gives the same results on all five puzzles while handling invalid input more reliably.

Execution Time: I added bitmasks to check whether a number can be placed using three bitmask lookups instead of scanning the row, column, and block. I also added the minimum remaining values (MRV) strategy, which selects the empty cell with the fewest possible numbers first. This can reduce the number of recursive calls needed to solve a puzzle. However, I would not claim a measured speedup on these five puzzles because both versions finish in about 2.5 ms.

Space Complexity: I changed the program to print each solution immediately instead of storing all solutions in a vector. I also removed the candidate vector that was created at every recursive call, so each call only needs a few integers on the stack. This reduces unnecessary memory use while still allowing the program to print every solution.

Maintainability: I kept the Board, SudokuSolver, and PuzzleRunner classes separate so each class has a clear responsibility. I added a LoadStatus enum to distinguish between file errors, invalid formats, conflicting numbers, and valid puzzles. I also added command-line arguments so users can choose puzzle files without changing the code, along with comments and a prologue comment block to make the program easier to understand.